Conversación con Claude — TPI MS2: diagnóstico (Fase 1)

Integrante: Cristian Morales
Herramienta: Claude (Anthropic), sesión de trabajo en claude.ai
Período: 17/10/2026 22:11 a 19/10/2026 21:57 (hora de Argentina) — 36 mensajes
Fuente: exportación oficial de datos de la cuenta (Settings → Privacy → Export data), conversación «Sistema de evaluación y setup»
Nota: se transcribe el texto visible de cada mensaje. Se omiten el razonamiento interno del modelo y el detalle técnico de las herramientas; en su lugar se indica la acción realizada (archivos creados, comandos ejecutados, archivos enviados). Las capturas de pantalla adjuntas están en la carpeta de evidencias del repositorio. Los mensajes posteriores a la exportación no están incluidos.
Parte: 2 de 2 — Diagnóstico de la parte asignada (líneas 1–111): revisión del análisis inicial, ejecución de casos, evaluación de resultados y ficha de smells. Viene de «cristian_claude_01_preparacion.pdf». Los últimos mensajes tratan sobre cómo exportar esta conversación.

Usuario (Cristian Morales) 17/10/2026 22:11

bueno bien, de la parte que me toca a Mi, Cristian, ayudame a hacer el diagnostrico

Claude 17/10/2026 22:11

Te ayudo a encontrarlos sin pasarte la lista armada. Si los encontrás vos, después los podés defender sin problema. La idea: te doy qué mirar y qué preguntarte en cada bloque, vos anotás lo que encontrás, y después lo revisamos juntos.

Tu parte son las **líneas 1 a 111** de `main.py`.

Paso 1: calidad externa (lo que ve el usuario)

Levantá el sistema y probá estos casos desde `/docs`. Para cada uno, anotá **el código de estado HTTP** y **la respuesta**:

#	Endpoint	Body	¿Qué esperarías?
1	`POST /estudiantes`	`{"nombre":"Ana","apellido":"Paz","legajo":"TUP-2001","email":"ana@ejemplo.com"}`	Alta correcta
2	`POST /estudiantes`	sin `apellido`	Error
3	`POST /estudiantes`	mismo `legajo` que el caso 1	Error
4	`POST /estudiantes`	`"email": "@"`	Error
5	`POST /estudiantes`	`"nombre": null`	Error
6	`POST /estudiantes`	`"apellido": null`	Error
7	`POST /responsables`	`"area": "Sistemas"`	Error
8	`POST /responsables`	dos veces el mismo responsable	¿Qué debería pasar?
9	`GET /estudiantes` y `GET /responsables`	—	Listados

Preguntas para cada resultado:

- ¿El código HTTP es coherente con lo que pasó? Por ejemplo, ¿un error devuelve 200, 400, 422 o 500?
- ¿Los casos 5 y 6 se comportan igual? ¿Deberían?
- ¿El caso 4 debería ser aceptado?
- ¿Un cliente de la API podría distinguir un éxito de un error sin leer el contenido?

Paso 2: calidad interna (bloque por bloque)

Líneas 7–8: la conexión

- ¿Dónde se crea la conexión y quién la usa? Buscá cuántas funciones usan `c` y `conn`.
- Si Bruno quiere que los tests usen una base distinta, ¿qué tendría que cambiar?
- ¿Qué pasa si dos requests llegan al mismo tiempo y usan el mismo cursor?

- Si mañana pasan de SQLite a PostgreSQL, ¿cuántos lugares del código hay que tocar?

Líneas 10–19: las tablas

- ¿En qué momento se ejecuta este código? ¿Qué pasa con solo hacer `import main`?
- Mirá los tipos de columna: ¿cómo se guardan las fechas?, ¿y el campo `activo`?
- ¿Hay algo que impida guardar una solicitud con un `estudiante_id` que no existe?

Líneas 21–54: los datos de prueba

- ¿Este código pertenece al sistema o es otra cosa? ¿Tiene sentido que viva en el mismo archivo que la API?
- ¿Qué condición decide si se cargan? ¿Qué pasa si hay estudiantes pero no hay solicitudes?
- El historial y las solicitudes se cargan por separado: ¿qué asegura que sean coherentes entre sí?

Líneas 59–76: `crear_estudiante`

- ¿Qué tipo tiene el parámetro `data`? ¿Qué te da FastAPI si usás otra cosa?
- Compará cómo se valida `nombre` con cómo se valida `apellido`. ¿Es consistente?
- ¿Cuántas veces aparece `data.get("carrera", "TUP")`? ¿Y `data.get("email", "")`?
- ¿Cuántas responsabilidades distintas tiene esta función? Contalas.

Líneas 79–86 y 105–111: los listados

- ¿Qué significan `r[3]` o `r[4]` sin mirar el `CREATE TABLE`?
- ¿Qué pasa con estos listados si alguien agrega una columna en medio de la tabla?
- ¿Esa conversión de fila a diccionario aparece en otros lugares del archivo?

Líneas 91–102: `nuevoResponsable`

- Compará los nombres de tus 4 funciones entre sí. ¿Siguen la misma convención?
- La lista de áreas válidas, ¿aparece en algún otro lugar del archivo? Si se agrega un área nueva, ¿cuántos lugares hay que tocar?
- La validación del email, ¿es igual a la de estudiantes?

Paso 3: ponerle nombre a lo que encontraste

Comparalo con el catálogo de smells de la materia. Los más comunes son: *Duplicated Code*, *Long Method*, *Large Class / God Object*, *Magic Numbers*, *Primitive Obsession*, *Global State*, *Inconsistent Naming*, *Shotgun Surgery* y *Feature Envy*. Para cada uno pensá también qué principio SOLID o qué práctica de Clean Code viola.

Paso 4: completar la ficha

Por cada hallazgo:

Campo	Ejemplo de qué poner
Smell	Nombre del catálogo
Evidencia	El fragmento de código
Ubicación	Función y líneas
Impacto	Qué se complica cambiar o probar
Riesgo	Qué podría romperse en el futuro
Prioridad	Alta, media o baja, con justificación

En tu parte hay bastante más que los 3 smells mínimos, y algunos afectan a todo el sistema, no solo a tu código. Esos son los "transversales" que después te toca resolver primero.

Cuando tengas tu lista, aunque sea en borrador, pasámela y la revisamos: te digo si algo está mal clasificado, si te falta evidencia o si se te escapó algo importante.

Usuario (Cristian Morales) 18/10/2026 12:20

Claude 18/10/2026 12:21

Ejecutó un comando: Verificar los casos del diagnóstico contra el sistema real

Lo revisé y corrí los 9 casos contra el sistema real. Antes del detalle, hay algo importante.

-
-

Las predicciones son correctas

Los 9 casos dan exactamente lo que dice el análisis: todos devuelven HTTP 200, incluidos los errores. Igual **corrélos vos y capturalos**, porque la consigna pide evidencia de funcionamiento, no predicciones.

Probando, encontré casos que el análisis no vio y que te conviene agregar a la tabla del Paso 1:

Caso	Resultado	Por qué importa
`"email": null`	**HTTP 500**, el servidor se rompe	Es el único caso que crashea. Es la evidencia más fuerte del manejo de errores inconsistente
`"legajo": null` dos veces	Se insertan **los dos**	El chequeo de legajo repetido no funciona con `null`, y no hay `UNIQUE` en la base que lo frene
Body que es una lista `[1,2]`	HTTP 422	Lo único que valida FastAPI por su cuenta es que el body sea un diccionario

Con el del 500 tenés un **defecto comprobado** y no solo un riesgo leído en el código. Esa es justo la distinción que el propio análisis te recomienda hacer.

Lo que corregiría en la ficha

1. Falta el hallazgo más visible. Que los errores vuelvan como HTTP 200 está bien explicado en el Paso 1, pero no aparece en la ficha final. Debería ser una fila propia, de prioridad alta:

- Smell: manejo de errores inconsistente. Se devuelve `{"error": ...}` con 200, mientras `getSolicitud` sí lanza 404 y el email `null` produce un 500.
- Evidencia: líneas 62, 64, 66, 68, 71, 94, 96 y 98.
- Impacto: un cliente no puede distinguir éxito de error por el código HTTP.

2. "Long Method" en `crear_estudiante` es débil. Son 18 líneas, y el propio análisis lo reconoce. Presentalo como **violación de SRP** (la función tiene 6 responsabilidades) y no como Long Method, porque si el docente cuenta las líneas, el argumento se cae.

3. La fila 4 mezcla cosas. "Estados de validación" no significa nada concreto. Separala en dos:

- **Falta de modelo de entrada:** `data: dict` en lugar de un esquema. Esto explica los casos 4, 5 y 6 y los nuevos.
- **Primitive Obsession:** las áreas como strings sueltos, y `activo` guardado como 0 o 1.

4. Los datos de prueba están mal clasificados. En las líneas 24 a 53 hay 30 `INSERT` casi idénticos. Eso es **Duplicated Code** concreto y visible, más fácil de defender que "riesgo de Shotgun Surgery".

5. Las filas 1 y 2 se solapan. Las dos describen estado global. O las fusionás, o dejás bien clara la diferencia: la 1 es el estado compartido que usan todas las funciones, y la 2 son los efectos secundarios de solo importar el módulo.

6. God Object va a la tabla grupal. Está en la tabla de SOLID pero no en tu ficha, y está bien así: es un problema de todo `main.py`, no solo de tu parte. Proponelo en la reunión de diagnóstico.

Siguiente paso

1. Corré los 9 casos y los 3 nuevos, y capturá cada resultado.
2. Ajustá la ficha con los 6 puntos de arriba.
3. Registrá el uso de IA.

Con eso tu parte queda completa para la puesta en común. Los hallazgos de conexión global y de importación con efectos secundarios son tus cambios transversales, así que llevalos a la reunión como prioridad.

Usuario (Cristian Morales) 18/10/2026 12:52

bien ayudame el paso a paso

Claude 18/10/2026 12:52

Va en tres pasos: evidencia, ficha y registro de IA. Calculá entre una hora y una hora y media en total.

Paso 1: correr los casos y capturarlos

1.1. Arrancar con la base limpia

Desde la carpeta del repo, en PowerShell:

```
git checkout dev
git pull
.\run.bat reset
.\run.bat
```

Abrí http://127.0.0.1:8000/docs.

1.2. Probar cada caso

En `/docs`, abrí el endpoint, tocá **Try it out**, pegá el body y tocá **Execute**. Corrélos **en este orden**, porque el caso 3 depende del 1 y el 8b del 8a:

#	Endpoint	Body
1	`POST /estudiantes`	`{"nombre":"Ana","apellido":"Paz","legajo":"TUP-2001","email":"ana@ejemplo.com"}`
2	`POST /estudiantes`	`{"nombre":"Ana","legajo":"TUP-2002","email":"ana@ejemplo.com"}`
3	`POST /estudiantes`	igual que el caso 1
4	`POST /estudiantes`	`{"nombre":"Ana","apellido":"Paz","legajo":"TUP-2004","email":"@"}`
5	`POST /estudiantes`	`{"nombre":null,"apellido":"Paz","legajo":"TUP-2005"}`
6	`POST /estudiantes`	`{"nombre":"Ana","apellido":null,"legajo":"TUP-2006"}`
7	`POST /responsables`	`{"nombre":"Laura Gil","area":"Sistemas"}`
8a	`POST /responsables`	`{"nombre":"Laura Gil","area":"Bedelia"}`
8b	`POST /responsables`	igual que el 8a
9	`GET /estudiantes` y `GET /responsables`	—
10	`POST /estudiantes`	`{"nombre":"Ana","apellido":"Paz","legajo":"TUP-2010","email":null}`
11a	`POST /estudiantes`	`{"nombre":"Ana","apellido":"Paz","legajo":null}`
11b	`POST /estudiantes`	igual que el 11a
12	`POST /estudiantes`	`[1, 2]`

1.3. Capturar

En cada captura tienen que verse el **body enviado**, el **código HTTP** y la **respuesta**. Guardalas así:

```
docs/evidencias/fase1/cristian/caso01_alta_ok.png
docs/evidencias/fase1/cristian/caso10_email_null_500.png
...
```

En el caso 10, además de la captura de `/docs`, capturá la **terminal donde corre uvicorn**. Ahí se ve el error (`TypeError`) con la línea exacta donde se rompe. Es la mejor evidencia del defecto.

1.4. Anotar el resultado real

Completá esta tabla con lo que **viste**, no con lo que predijo la IA:

#	HTTP obtenido	Respuesta	¿Es correcto?	Observación
1				
…				

Paso 2: ajustar la ficha

Partí de la ficha del análisis y aplicá estos cambios, **escribiéndolos con tus palabras**:

#	Hallazgo	Qué hacer
1	Estado global (conexión y cursor compartidos)	Dejar. Aclarar que es el estado que usan todas las funciones
2	Efectos secundarios al importar el módulo	Dejar. Diferenciarlo del 1: acá el problema es que *importar* modifica la base
3	**Manejo de errores inconsistente**	**Agregar, prioridad alta.** Evidencia: casos 2, 3, 5 y 7 (errores con 200) y el caso 10 (500)
4	Duplicated Code: validación de email	Dejar. Sumar el caso 10 como evidencia de que esa validación además es frágil
5	**Falta de modelo de entrada (`data: dict`)**	**Separar de la fila 4 original.** Evidencia: casos 4, 6 y 11
6	Primitive Obsession: áreas como strings y `activo` como 0 o 1	La otra mitad de la fila 4 original
7	**Violación de SRP en `crear_estudiante`**	**Reemplaza a "Long Method".** Listar las 6 responsabilidades
8	Magic Numbers: `r[0]` ... `r[5]`	Dejar
9	Duplicated Code: conversión de filas a diccionario	Dejar
10	**Duplicated Code: 30 `INSERT` en los datos de prueba**	**Reemplaza a "Shotgun Surgery"** en esa sección
11	Integridad de datos insuficiente	Dejar. Sumar los casos 8 y 11 como evidencia
12	Inconsistent Naming	Dejar, prioridad baja
13	Riesgo de Shotgun Surgery: áreas	Dejar como riesgo, prioridad media

Para cada fila completá las 6 columnas: smell, evidencia, ubicación, impacto, riesgo y prioridad. Agregá una séptima columna con la **referencia a la captura** cuando la haya. Así cada hallazgo se puede verificar.

Aparte, anotá para la reunión grupal:

- **God Object** (`main.py` completo), para proponerlo en la tabla grupal.
- Los hallazgos **1 y 2**, que son los transversales que te toca resolver primero.

Guardalo como `docs/diagnostico/cristian.md`.

Paso 3: registrar el uso de IA

1. Exportá o copiá los chats que usaste: el de la otra IA que generó el análisis y este.
2. Guardalos como `anexo/ia/cristian_diagnostico_<herramienta>.md` o en PDF.
3. Completá tu fila del registro:

Campo	Qué poner
Herramienta	El nombre de cada IA que usaste
Para qué	Análisis inicial de smells de las líneas 1–111. Revisión del análisis y detección de casos adicionales
Grado de intervención	Sé honesto. Por ejemplo: "La IA propuso la primera versión; verifiqué los casos ejecutándolos, corregí la clasificación y reescribí la ficha"
Evidencia	La ruta del archivo en el anexo

Paso 4: subirlo

```
git checkout -b docs/diagnostico-cristian
git add docs/
git commit -m "docs: diagnóstico de conexión, estudiantes y responsables"
git push -u origin docs/diagnostico-cristian
```

Abrí el PR contra `dev` y asignáselo a **Gaston**, que te revisa a vos y además coordina la tabla de smells.

Si en algún caso te da un resultado distinto al de la tabla, o el 500 no aparece, pasame la captura y lo vemos.

Usuario (Cristian Morales) 19/10/2026 21:21

te puedo poner las capturas aca asi me las vas armando?

Claude 19/10/2026 21:21

Sí, mandámelas acá. Con cada tanda hago esto:

- **Reviso que cada captura sirva como evidencia:** que se vean el body, el código HTTP y la respuesta. Si falta algo, te digo cuál repetir.
- **Completo la tabla de resultados** del Paso 1.4 con lo que muestra cada captura.
- **Te digo con qué nombre guardar cada una**, para que coincidan con la carpeta y con las referencias de la ficha.
- **Te marco** si algún resultado no da lo esperado.

Al final te paso la tabla armada en un `.md`, lista para pegar en `docs/diagnostico/cristian.md`.

Para que sea más fácil:

- Mandalas **en orden**, de a varias por mensaje.
- Si en el nombre del archivo o en el mensaje ponés qué caso es cada una ("caso 1", "caso 10 terminal"), no hay confusión.
- En el caso 10 mandá también la captura de la terminal de uvicorn.

Una aclaración: la tabla de resultados la armo yo, pero la ficha de smells conviene que la escribas vos con tus palabras. La reviso cuando la tengas.

Usuario (Cristian Morales) 19/10/2026 21:22

ahi se ve el body en el curl, esta bien no?

[1 captura de pantalla adjunta: ver docs/evidencias/fase1/cristian/]

Claude 19/10/2026 21:22

Sí, así está bien. En el `curl` se ve todo lo necesario:

- **Método y URL:** `POST /estudiantes`
- **Body enviado:** en la línea `-d`
- **Código HTTP:** 200
- **Respuesta:** el estudiante creado, con `id: 5`
- **Fecha y hora:** en los headers. Es un plus, porque prueba cuándo se corrió

Caso 1 registrado:

#	HTTP obtenido	Respuesta	¿Es correcto?	Observación
1	200	Devuelve el estudiante creado (id 5)	Parcial	El alta funciona, pero un recurso creado debería responder **201 Created**, no 200

Guardala como `caso01_alta_ok.png`.

El `id: 5` es esperable, porque los datos de prueba ya cargan 4 estudiantes. Confirma además que arrancaste con la base limpia.

Seguí con el mismo encuadre para el resto. En los casos de error (2, 3, 5 y 7) lo importante es que se vea que el código es **200** aunque la respuesta diga `"error"`.

Usuario (Cristian Morales) 19/10/2026 21:26

ahi estan en orden las capturas

[15 capturas de pantalla adjuntas: ver docs/evidencias/fase1/cristian/]

Claude 19/10/2026 21:27

Ejecutó un comando: Copiar las capturas con los nombres de evidencia

Creó el archivo `cristian_resultados_fase1.md`

Ejecutó un comando: Empaquetar evidencias y tabla de resultados

Envió el archivo `evidencias-cristian.zip`

Armé **evidencias-cristian.zip** con dos cosas, ya en la estructura del repo:

- **`docs/evidencias/fase1/cristian/`**: las 16 capturas, incluida la del caso 1 de antes, renombradas como `caso01_alta_ok.png`, `caso10_email_null_500.png` y así.
- **`docs/diagnostico/cristian_resultados_fase1.md`**: la tabla con el resultado de cada caso, la referencia a su captura y las conclusiones de calidad externa.

Todos los resultados coinciden con lo esperado, y el 500 del caso 10 quedó documentado.

Falta completar

1. **Caso 11a y 11b (`"legajo": null` dos veces).** La captura 14 es una repetición del caso 10: tiene el mismo body, `TUP-2010` con `email: null`. La guardé como prueba de que el error se repite, pero el caso 11 no está. Los bodies son:

   ```
   {"nombre":"Ana","apellido":"Paz","legajo":null}
   ```

   Mandalo dos veces y, después, un `GET /estudiantes`. Ahí tendrían que verse los dos registros con `legajo: null`.
2. **Terminal de uvicorn del caso 10.** Volvé a mandar ese caso y capturá la terminal: tiene que aparecer el `TypeError` con la línea del código donde falla. Es lo que demuestra **por qué** pasa el 500, no solo que pasa.

Con esas dos capturas completo las filas que faltan y cierro la tabla.

Usuario (Cristian Morales) 19/10/2026 21:29

ahi seria?

[2 capturas de pantalla adjuntas: ver docs/evidencias/fase1/cristian/]

Claude 19/10/2026 21:29

Ejecutó un comando: Agregar el caso 11 a las evidencias y la tabla

Envió el archivo evidencias-cristian.zip

Sí, con esas dos capturas el caso 11 queda probado. Actualicé **evidencias-cristian.zip**: suma las dos capturas y completa las filas 11a, 11b y 11c de la tabla.

El caso 11 queda así:

- **11a:** se guarda un estudiante con `legajo: null` (`id` 8).
- **11b:** se guarda un segundo estudiante igual (`id` 9), y el sistema no lo detecta como repetido.
- **11c:** el listado muestra los dos registros, idénticos.

No hay captura del POST de 11a, solo del 11b. Igual el `id` 8 aparece en el listado, así que tomé esa captura como su evidencia. Si querés tenerla igual, podés repetir el POST, aunque va a crear un `id` 10.

Para la defensa: el control de legajo repetido falla con `null` porque la consulta compara `legajo = NULL`, y en SQL esa comparación nunca da verdadero. Además, la tabla no tiene restricción `UNIQUE` que lo frene. Es un buen ejemplo de por qué la validación en el código no reemplaza la integridad en la base.

Falta solo la captura de la terminal de uvicorn del caso 10. Mandá de nuevo ese body, buscá el `TypeError` en la terminal y capturá desde el `Traceback` hasta la última línea. Esa captura muestra en qué línea del código se rompe, además de mostrar que se rompe.

Usuario (Cristian Morales) 19/10/2026 21:31

[2 capturas de pantalla adjuntas: ver docs/evidencias/fase1/cristian/]

Claude 19/10/2026 21:31

Ejecutó un comando: Agregar traceback del caso 10 y cerrar la tabla

Envió el archivo evidencias-cristian.zip

Con esas capturas quedó completa la evidencia de la Fase 1. **evidencias-cristian.zip** tiene las 20 capturas y la tabla cerrada.

El traceback confirma la causa del 500:

```
main.py, line 67, in crear_estudiante
    if "email" in data and "@" not in data["email"]:
TypeError: argument of type 'NoneType' is not a container or iterable
```

La validación revisa que exista la clave `email`, pero no que su valor sea un texto. Con `null` intenta buscar `@` dentro de `None`, y como no hay manejo de errores, la excepción llega sin controlar y el servidor responde 500. Lo agregué a la fila 10 y a las conclusiones.

Al final de la tabla sumé una sección **"Vínculo con la ficha de smells"** que cruza cada hallazgo con los casos que lo prueban. Te sirve para completar la columna de evidencia de la ficha.

Lo que te queda

1. **Escribir la ficha de smells** con tus palabras, aplicando los ajustes que vimos. Pasámela y la reviso.
2. **Completar el registro de IA** con los chats.
3. **Subir todo** en la rama `docs/diagnostico-cristian` y abrir el PR a `dev` con Gaston como revisor.

Descomprimí el zip en la raíz del repo: las carpetas `docs/evidencias/...` y `docs/diagnostico/...` ya quedan en su lugar.

Usuario (Cristian Morales) 19/10/2026 21:32

me pasas una tabla con los casos y que responde y al lado pongo si esta bien o no?

Claude 19/10/2026 21:32

Acá va. Las columnas **¿Está bien?** y **¿Por qué?** quedan vacías para que las completes vos:

#	Qué se probó	Body enviado	HTTP	Qué respondió	¿Está bien?	¿Por qué?
1	Alta de estudiante válido	Datos completos, legajo `TUP-2001`	200	Estudiante creado (id 5)		
2	Alta sin apellido	Sin el campo `apellido`	200	`{"error":"falta apellido"}`		
3	Legajo repetido	Mismo body que el caso 1	200	`{"error":"legajo repetido"}`		
4	Email inválido	`"email":"@"`	200	Estudiante creado (id 6) con email `"@"`		
5	Nombre nulo	`"nombre":null`	200	`{"error":"falta nombre"}`		
6	Apellido nulo	`"apellido":null`	200	Estudiante creado (id 7) con `apellido: null`		
7	Área inexistente	`"area":"Sistemas"`	200	`{"error":"area invalida"}`		
8a	Alta de responsable	`Laura Gil, Bedelia`	200	Responsable creado (id 6)		
8b	Responsable repetido	Mismo body que el 8a	200	Responsable creado (id 7)		
9a	Listado de estudiantes	—	200	7 estudiantes, incluidos los de los casos 4 y 6		
9b	Listado de responsables	—	200	7 responsables, con el 6 y el 7 idénticos		
10	Email nulo	`"email":null`	500	`Internal Server Error`		
11a	Legajo nulo	`"legajo":null`	200	Estudiante creado (id 8) con `legajo: null`		
11b	Legajo nulo repetido	Mismo body que el 11a	200	Estudiante creado (id 9) con `legajo: null`		
12	Body que no es un objeto	`[1, 2]`	422	`"Input should be a valid dictionary"`		

Para cada caso, revisá dos cosas por separado:

- **¿El resultado es correcto?** Por ejemplo, si se creó o se rechazó lo que correspondía.
- **¿El código HTTP es correcto?** Pensá qué código debería devolver: 201, 400, 409, 422 o 500.

Un caso puede tener bien el resultado y mal el código. Cuando la completes, pasámela y la comparo con la que armé.

Usuario (Cristian Morales) 19/10/2026 21:37

1, parcial, deberia ser un 201 created
2 parcial, esta bien el error, pero creo que deberia devolver 400 o 422 creo
3 lo mismo que 2
4 mal, falta checkeo de mail correcto, deberia devolver un 400/422
5 lo mismo que 2
6 esta mal, no deberia crear sin apellido y devolver un 400/422 (con mensaje de error)
7 lo mismo que 2
8 lo mismo que 1
9 mal, no verifica dupluicado deberia tirar un 409 con mensaje de error

10 mal, no esta especifico que sea requerido el mail, pero hay error de tipado creo
11a mal, no deberia dejar crear con legajo nulo (dato requerido)
11b lo mismo que 9
12 creo que esta bien el codigo de error, no esta bien el mensaje (o podria ser mas claro)

Claude 19/10/2026 21:38

Muy bien encaminado: acertaste el diagnóstico en casi todos. Hay tres ajustes y un tema de numeración.

Ajustes

Caso 3 (legajo repetido): no es igual al 2. El caso 2 es un dato mal enviado, mientras que el 3 es un dato válido que **choca con uno que ya existe**. Para eso se usa **409 Conflict**, el mismo código que propusiste para el responsable duplicado. Si en el 3 ponés 400/422 y en el duplicado 409, la tabla queda incoherente.

Numeración de los casos 8 y 9. Tu "8" es el **8a** y tu "9" es el **8b** (responsable duplicado → 409, bien). Te faltan los listados:

- **9a y 9b: están bien.** Devuelven 200 con los datos, que es lo correcto para un GET. Lo que muestran está mal, pero eso viene de los casos 4, 6 y 8b, no del listado. Sirven como **evidencia** de que esos datos inválidos quedaron guardados.

Caso 10: tu razonamiento va bien, falta el punto principal. Es cierto que el email no es obligatorio y que el error es de tipo (`TypeError`). Lo más importante es otra cosa: **un 500 nunca debería producirse por un dato que manda el cliente**. El 500 significa "falló el servidor". Con `email: null`, el sistema debería aceptarlo como "sin email" o rechazarlo con 422, pero no romperse.

Dos detalles para sumar

- **Caso 11a:** agregá el código, igual que en el 6: debería rechazarse con **422**.
- **Caso 12:** está bien lo que viste del mensaje, y hay algo más: el formato del error es distinto al del resto. Acá responde `{"detail": [...]}` y en los demás `{"error": "..."}`. Un cliente tendría que manejar **dos formatos de error distintos** en el mismo endpoint, y eso refuerza el hallazgo de manejo de errores inconsistente.

Sobre 400 o 422

Elijan uno y úsenlo en todo el sistema. Como convención con FastAPI:

Situación	Código
Recurso creado	201
Dato inválido o faltante	422
Choca con un dato existente (duplicado)	409
El recurso no existe	404
Falla interna	500, nunca por culpa de un dato del cliente

Sirve que la tabla de códigos esté en `docs/` como **decisión técnica**, así todo el grupo usa los mismos.

Tu tabla, con los ajustes

#	¿Está bien?	¿Por qué?
1	Parcial	Crea bien, pero debería responder 201 Created
2	Parcial	El error es correcto, pero debería responder 422 en vez de 200
3	Parcial	Detecta el duplicado, pero debería responder 409 Conflict en vez de 200
4	No	No valida el formato del email; debería rechazarlo con 422
5	Parcial	Igual que el 2
6	No	Crea un estudiante sin apellido; debería rechazarlo con 422 y un mensaje de error
7	Parcial	Igual que el 2
8a	Parcial	Igual que el 1

#	¿Está bien?	¿Por qué?
8b	No	No verifica duplicados; debería responder 409 con un mensaje de error
9a	Sí	El listado funciona; evidencia que se guardaron los datos inválidos de los casos 4 y 6
9b	Sí	El listado funciona; evidencia el duplicado del 8b
10	No	Un dato del cliente rompe el servidor (`TypeError` en la línea 67). Debería aceptar el email nulo o rechazarlo con 422, nunca responder 500
11a	No	Crea un estudiante sin legajo, que es obligatorio; debería rechazarlo con 422
11b	No	Igual que el 8b: no detecta el duplicado y debería responder 409
12	Sí	El código 422 es correcto, pero el mensaje es poco claro y el formato (`detail`) no coincide con el del resto de los errores (`error`)

Agregá estas dos columnas a la tabla de `cristian_resultados_fase1.md`, en lugar de la columna "¿Correcto?" que puse yo. Así queda tu evaluación y no la mía.

Usuario (Cristian Morales) 19/10/2026 21:39

armame todo junto a las capturas para descargar

Claude 19/10/2026 21:40

Creó el archivo `cristian_resultados_fase1.md`

Leyó el archivo `cristian_resultados_fase1.md`

Creó el archivo `cristian_resultados_fase1.md`

Ejecutó un comando: Reempaquetar evidencias con la tabla final

Envió el archivo `evidencias-cristian.zip`

Te dejé todo en **evidencias-cristian.zip**. Al descomprimirlo en la raíz del repo queda así:

```
docs/
├── diagnostico/
│   └── cristian_resultados_fase1.md
└── evidencias/fase1/cristian/
    └── 20 capturas (caso01_alta_ok.png … caso12_body_lista_422.png)
```

El `.md` tiene:

- **Criterio de códigos HTTP**: la tabla de 201, 422, 409, 404 y 500 que usaste para evaluar.
- **Tabla de resultados** con tus columnas "¿Está bien?" y "¿Por qué?", y la referencia a la captura de cada caso. Reemplacé las columnas que había puesto yo.
- **Detalle del caso 10**, con el traceback y la explicación.
- **Conclusiones de calidad externa**, a las que sumé los dos formatos de error que viste en el caso 12.
- **Vínculo con la ficha de smells**, para la columna de evidencia de la ficha.

Ahora te queda escribir la ficha de smells.

Usuario (Cristian Morales) 19/10/2026 21:43

ayudame con la ficha de smells

Claude 19/10/2026 21:43

Lo hacemos igual que con la tabla de resultados. Te paso la ficha con las columnas fácticas ya completas (smell, ubicación, evidencia y casos), y vos completás las de análisis: **impacto, riesgo, prioridad y principio**. Esas son las que tenés que poder defender.

La ficha

#	Smell / hallazgo	Ubicación	Evidencia (código)	Casos	Impacto	Riesgo	Prioridad	Principio
1	Global State	Líneas 7–8	`conn = sqlite3.connect("solicitudes.db",`	—				

#	Smell / hallazgo	Ubicación	Evidencia (código)	Casos	Impacto	Riesgo	Prioridad	Principio
			`check_same_thread=False)` y `c = conn.cursor()`, usados por todas las funciones					
2	Efectos secundarios al importar el módulo	Líneas 10–54	`CREATE TABLE` e `INSERT` sueltos a nivel de módulo, fuera de cualquier función	—				
3	Manejo de errores inconsistente	Líneas 62, 64, 66, 68, 71, 94, 96 y 98	`return {"error": "..."}` en lugar de lanzar una excepción HTTP	2, 3, 5, 7, 10, 12				
4	Duplicated Code: validación de email	Líneas 67–68 y 97–98	`if "email" in data and "@" not in data["email"]:` repetido en los dos endpoints	4, 10				
5	Falta de modelo de entrada	Líneas 60 y 92	`data: dict = Body(...)`, con validaciones manuales campo por campo	4, 6, 10, 11a, 12				
6	Primitive Obsession	Líneas 95, 100 y 110	Áreas como lista de strings; `activo` guardado como `1/0` y convertido con `r[4] == 1`	7				
7	Violación de SRP en `crear_estudiante`	Líneas 59–76	Valida campos, valida email, verifica duplicados, inserta, hace commit y arma la respuesta	—				
8	Magic Numbers	Líneas 85 y 110	`r[0]`, `r[1]` … `r[5]` para leer columnas	—				
9	Duplicated Code: conversión de filas	Líneas 83–86 y 108–110	Mismo bucle `for r in ...: res.append({...})` en los dos listados	—				
10	Duplicated Code: datos de prueba	Líneas 24–53	30 `INSERT` casi idénticos con los valores escritos a mano	—				
11	Integridad de datos insuficiente	Líneas 10–18 y 69	Sin `NOT NULL`, `UNIQUE` ni claves foráneas; `WHERE legajo = ?` no detecta `NULL`	6, 8b, 11a, 11b				
12	Inconsistent Naming	Líneas 60, 80, 92 y 106	`crear_estudiante`, `listar_estudiantes`, `nuevoResponsable`, `get_resp`	—				
13	Riesgo de Shotgun Surgery: áreas	Línea 95 (y su uso en `registrar`, líneas 167–174)	Las áreas válidas están escritas en un lugar y el mapeo de tipo a área en otro	—				

Cómo completar cada columna

Impacto: qué se complica hoy. Respondé: "*¿Qué cuesta cambiar, probar o entender por culpa de esto?*" Por ejemplo, "para testear con otra base hay que modificar el código".

Riesgo: qué puede romperse mañana. Respondé: "*Si alguien toca esto, ¿qué falla puede aparecer?*" El impacto es el costo actual y el riesgo es la falla futura. No repitas lo mismo en las dos columnas.

Prioridad: alta, media o baja, siempre con una justificación corta. Tres criterios que te sirven:

- **¿Ya produce un defecto comprobado?** Si hay un caso que lo prueba, sube.
- **¿Afecta a todo el sistema o solo a una función?** Si es transversal, sube.
- **¿Bloquea otros refactorings o los tests?** Si los bloquea, va primero.

Principio: qué principio SOLID o qué práctica de Clean Code viola. Las opciones más comunes son SRP, OCP, DIP, DRY, nombres expresivos y manejo de errores. No fuerces un principio SOLID si no corresponde: en varias filas la respuesta correcta es una práctica de Clean Code.

Dos pistas

- **Las filas 1 y 2 parecen iguales, pero no lo son.** Pensá qué pasa en cada caso cuando Bruno quiera correr los tests con una base limpia.
- **Hay hallazgos que se encadenan.** Por ejemplo, la 5 explica en parte por qué existe la 4. Si lo mencionás en el impacto, el análisis gana mucho.

Completala como hiciste con la anterior, aunque sea en borrador o fila por fila, y la revisamos.

Usuario (Cristian Morales) 19/10/2026 21:46

1. Casos: —, Impacto: dificulta probar con otra base de datos, Riesgo: conflictos entre operaciones concurrentes, Prioridad: alta, porque afecta a todo el sistema, Principio: DIP.
2. Casos: —, Impacto: importar el módulo modifica la base de datos y dificulta los tests, Riesgo: datos duplicados o modificaciones inesperadas al importar, Prioridad: alta, porque complica las pruebas, Principio: SRP.
3. Casos: 2, 3, 5, 7, 10, 12, Impacto: obliga al cliente a interpretar errores con respuestas inconsistentes, Riesgo: errores respondidos con HTTP 200 en lugar del estado correcto, Prioridad: alta, porque ya afecta el comportamiento de la API, Principio: manejo de errores.
4. Casos: 4, 10, Impacto: hay que modificar dos lugares para cambiar la validación del email, Riesgo: que los endpoints validen de manera diferente, Prioridad: media, porque duplica una regla de negocio, Principio: DRY.
5. Casos: 4, 6, 10, 11a, 12, Impacto: las validaciones manuales aumentan el código y dificultan mantenerlo; favorecen la duplicación de la fila 4, Riesgo: aceptar datos inválidos o generar errores inesperados, Prioridad: alta, porque afecta la entrada de datos en varios endpoints, Principio: Clean Code: validación y modelos explícitos.
6. Casos: 7, Impacto: cuesta entender qué representa cada string y el valor `1` o `0`, Riesgo: errores al agregar áreas o interpretar el estado activo, Prioridad: media, porque dificulta extender y mantener los datos, Principio: Clean Code: tipos expresivos.
7. Casos: —, Impacto: cuesta modificar y probar por separado las responsabilidades de la función, Riesgo: un cambio en una tarea puede afectar las demás, Prioridad: alta, porque concentra varias responsabilidades, Principio: SRP.
8. Casos: —, Impacto: cuesta entender qué columna representa cada índice, Riesgo: intercambiar índices y devolver datos incorrectos al cambiar la consulta, Prioridad: media, porque dificulta mantener los listados, Principio: Clean Code: nombres expresivos.
9. Casos: —, Impacto: hay que mantener dos conversiones similares, Riesgo: que los listados devuelvan formatos diferentes después de un cambio, Prioridad: media, porque una función compartida simplificaría el mantenimiento, Principio: DRY.
10. Casos: —, Impacto: cuesta editar y mantener los datos de prueba escritos manualmente, Riesgo: inconsistencias o errores al modificar los registros, Prioridad: baja, porque afecta principalmente al mantenimiento de los datos iniciales, Principio: DRY.
11. Casos: 6, 8b, 11a, 11b, Impacto: la base permite datos incompletos o duplicados, Riesgo: registros inválidos, duplicados o relaciones inconsistentes, Prioridad: alta, porque puede comprometer la integridad de los datos, Principio: integridad de datos y validación.
12. Casos: —, Impacto: los nombres inconsistentes dificultan localizar y entender las funciones, Riesgo: confusiones al usar o modificar endpoints, Prioridad: baja, porque es principalmente un problema de legibilidad, Principio: Clean Code: nombres consistentes.
13. Casos: —, Impacto: para cambiar las áreas válidas hay que revisar varios lugares del código, Riesgo: que el mapeo y la lista de áreas queden desincronizados, Prioridad: media, porque complica los cambios relacionados, Principio: OCP y DRY.

Claude 19/10/2026 21:46

Muy buena ficha: las 13 filas tienen las cinco columnas completas, y en la 5 encadenaste con la 4, que era la pista. Hay cinco ajustes y uno de criterio general.

Ajustes

Fila 2: el riesgo no es exacto. Los datos de prueba solo se cargan si no hay estudiantes (línea 23), así que importar dos veces **no duplica**. El riesgo real es otro: cualquier `import main`, por ejemplo desde los tests, **crea o modifica el archivo `solicitudes.db` real**. Así los tests escriben sobre la base de trabajo y dependen de lo que dejó la corrida anterior.

Esta es la diferencia con la fila 1: la 1 trata de **quién usa** la conexión, y la 2 de **qué pasa con solo importar**.

Fila 3: el riesgo repite el estado actual. "Errores con HTTP 200" ya pasa hoy, así que es parte del impacto. El riesgo es la consecuencia futura. Por ejemplo: un frontend que mira el código HTTP muestra "estudiante creado"

cuando en realidad falló, y las excepciones no controladas siguen apareciendo como 500, como en el caso 10.

Fila 7: la prioridad alta no está justificada. Con tus propios criterios, no es transversal (es una sola función) y no produce un defecto por sí misma. Los defectos que tiene ya están en las filas 3, 4 y 5. Sugiero **media**. Si la querés dejar alta, la justificación tiene que ser otra, porque "concentra varias responsabilidades" describe el smell, no por qué es urgente.

Fila 11: el principio no es un principio. "Integridad de datos y validación" describe el problema. Tenés dos opciones:

- Aclarar que es un **hallazgo de diseño de la base**, no un smell del catálogo. Es lo más honesto y suma puntos.
- O vincularlo con **"fail fast" / validar en el borde del sistema**, que es una práctica de Clean Code.

Agregá también el caso **9b** a la columna de casos, porque es el listado que muestra el duplicado.

Fila 6: el caso 7 no prueba el smell. En ese caso, la validación de área **funciona**. Dejá la columna de casos vacía, o explicá que el caso muestra que el área se valida contra strings escritos a mano.

Criterio general: demasiadas prioridades altas

Tenés 6 de 13 en alta. Si casi todo es alto, la priorización no ordena nada. Con el cambio de la fila 7 quedan 5, y se justifican porque son las que producen defectos comprobados o bloquean los tests:

Prioridad	Filas	Por qué van ahí
Alta	1, 2, 3, 5, 11	Bloquean los tests (1 y 2) o tienen defectos comprobados (3, 5 y 11)
Media	4, 6, 7, 8, 9, 13	Dificultan mantener el código, sin falla actual
Baja	10, 12	Legibilidad y datos de prueba

Esta tabla te sirve además para decir en la reunión **en qué orden resolverlos**: primero 1 y 2, que son tus transversales y desbloquean los tests de Bruno.

Pasame las filas 2, 3, 6, 7 y 11 corregidas con tus palabras y te armo el `.md` final de la ficha para sumar al zip.

Usuario (Cristian Morales) 19/10/2026 21:47

2. Casos: —, Impacto: importar el módulo modifica la base de datos y dificulta los tests, Riesgo: los tests pueden modificar la base real y depender de datos de ejecuciones anteriores, Prioridad: alta, porque bloquea pruebas confiables, Principio: SRP / evitar efectos secundarios al importar.
3. Casos: 2, 3, 5, 7, 10, 12, Impacto: el cliente recibe respuestas de error con estados HTTP incorrectos, Riesgo: el frontend puede interpretar un fallo como una operación exitosa y las excepciones no controladas pueden generar errores 500, Prioridad: alta, porque hay defectos comprobados, Principio: manejo de errores.
4. Casos: —, Impacto: cuesta entender qué representan los strings de áreas y los valores `1` y `0`, Riesgo: errores al ampliar las áreas o interpretar el estado activo, Prioridad: media, porque dificulta el mantenimiento, Principio: Clean Code: tipos y nombres expresivos.
5. Casos: —, Impacto: cuesta modificar y probar por separado las tareas de la función, Riesgo: un cambio en una responsabilidad puede afectar las demás, Prioridad: media, porque dificulta el mantenimiento, pero no genera por sí sola un defecto comprobado, Principio: SRP.
6. Casos: 6, 8b, 9b, 11a, 11b, Impacto: la base permite datos incompletos, duplicados o relaciones inconsistentes, Riesgo: que esos datos inválidos provoquen consultas incorrectas o errores en operaciones futuras, Prioridad: alta, porque hay defectos comprobados, Principio: diseño de base de datos: integridad de datos; práctica relacionada: fail fast.

Claude 19/10/2026 21:48

Creó el archivo `cristian_ficha_smells.md`

Ejecutó un comando: Sumar la ficha al zip de evidencias

Envió el archivo `evidencias-cristian.zip`

Las cinco filas corregidas están bien. Ya está lista la ficha, en **evidencias-cristian.zip**, junto con la tabla de resultados y las capturas.

`docs/diagnostico/cristian_ficha_smells.md` tiene:

- **Resumen:** los 13 hallazgos con ubicación, casos y prioridad en una sola tabla, fácil de llevar a la tabla grupal de Gaston.

- **Criterio de priorización** y **orden de resolución propuesto**.
- **Detalle de cada hallazgo** con tus textos tal como los escribiste. Los armé en secciones en lugar de una tabla de 9 columnas, que en GitHub se ve apretada.
- **Para la puesta en común:** la propuesta de God Object, los transversales 1 y 2, y un aviso para Gaston. El hallazgo 13 toca su código (`registrar`), así que conviene que lo coordinen.

Te faltan dos cosas:

1. **Registro de IA**: guardá los chats en el anexo y completá tu fila.
2. **Subir todo:** descomprimí el zip en la raíz del repo y corré:

```
git checkout dev
git pull
git checkout -b docs/diagnostico-cristian
git add docs/
git commit -m "docs: diagnóstico de conexión, estudiantes y responsables"
git push -u origin docs/diagnostico-cristian
```

Después abrí el PR a `dev` y asigná a Gaston como revisor.

Usuario (Cristian Morales) 19/10/2026 21:53

puede ser que falto poner el tag en el commit?

Claude 19/10/2026 21:53

No, en tu commit todavía no va el tag.

El tag `etapa-diagnostico` marca el **cierre de la fase para todo el grupo**, no el aporte de cada uno. Se pone una sola vez, cuando:

1. Los cuatro diagnósticos ya están mergeados en `dev`.
2. Gaston unificó la tabla grupal de smells y también está mergeada.
3. Hicieron la reunión de puesta en común.

Si lo pusieras ahora, el tag marcaría un `dev` que solo tiene tu parte, y la fase quedaría registrada como cerrada sin estarlo.

Cuando cierren la fase, lo pone quien coordina los tags (sos vos, según el plan):

```
git checkout dev
git pull
git tag etapa-diagnostico
git push origin etapa-diagnostico
```

Si te referías al **prefijo del mensaje** del commit, ese ya está: `docs:` en `"docs: diagnóstico de conexión, estudiantes y responsables"`.

Usuario (Cristian Morales) 19/10/2026 21:55

como puedo bajar la conversacion?

Claude 19/10/2026 21:55

Búsqueda web: Claude export conversation download chat support.claude.com

Consultó la página https://support.claude.com/en/articles/9450526

Tenés dos formas, según lo que necesites.

1. Exportación oficial de Claude (todas tus conversaciones juntas)

Desde la web o la app de escritorio, no desde el celular:

1. Tocá tus iniciales, abajo a la izquierda.
2. Entrá a **Settings** → **Privacy**.
3. Tocá **Export data**.

Te llega un mail con un link de descarga. Tenés que estar logueado para bajarlo y el link vence a las 24 horas. Ojo: exporta **todo tu historial y los datos de la cuenta**, no un chat suelto. Para el anexo te va a sobrar contenido, y puede tener chats personales que no corresponde entregar.

2. Guardar solo este chat como PDF (lo que te conviene para el anexo)

Abrí esta conversación en el navegador, apretá **Ctrl + P** y elegí **Guardar como PDF** en la impresora. Es una función del navegador, no de Claude, así que puede cortar algún bloque largo o algún formato. Revisá el PDF antes de subirlo.

Guardalo como `anexo/ia/cristian_claude_diagnostico.pdf` y hacé lo mismo con el chat de la otra IA que usaste para el primer análisis.

Fuentes:

- Claude Help Center — How can I export my Claude data?
- AI Toolbox — How to Save Claude as PDF

Usuario (Cristian Morales) 19/10/2026 21:57

como abro la conversaicon en el navegador?

Claude 19/10/2026 21:57

Esta conversación tiene este link:

https://claude.ai/code/session_01GBiL9SrpXMXSvHWetbYgrR

Copialo en Chrome, Edge o el navegador que uses, con la misma cuenta con la que estás ahora. Si te pide iniciar sesión, entrá con tu usuario de Claude.

Otra opción: entrá a **claude.ai** y buscá la conversación en la lista de recientes de la barra lateral.

Una vez abierta, hacé **Ctrl + P** → **Guardar como PDF**. Como la conversación es larga, puede que no carguen todos los mensajes de una vez. Antes de imprimir, subí hasta el principio para que se carguen todos, y revisá que el PDF empiece con el primer mensaje, el del resumen del TP.

Usuario (Cristian Morales) 19/10/2026 21:57

me deja todo en blanco al hacer ctrl p

Claude 19/10/2026 21:57

Pasa con algunas páginas que muestran el chat dentro de un panel con scroll propio: el navegador imprime la página, pero no el contenido del panel. Dos alternativas que funcionan:

Opción 1: copiar y pegar (la más rápida)

1. Subí hasta el primer mensaje de la conversación, para que se cargue entera.
2. Hacé clic al principio del primer mensaje, bajá hasta el final y hacé **Shift + clic** al final del último. Así seleccionás todo el chat.
3. **Ctrl + C**, pegalo en un documento de Word o Google Docs y guardalo como PDF.

Se pierde algo del formato, pero queda todo el texto, que es lo que importa como evidencia. Las imágenes que mandaste no se copian, pero ya están en la carpeta de evidencias.

Opción 2: la exportación oficial

La de **Settings → Privacy → Export data** que te pasé antes. Trae todo el historial, así que después tenés que separar este chat del resto.

Si ninguna te funciona, avisame y probamos otra cosa.